

MECHATRONICS AND IOT

ASSIGNMENT REPORT – 12

TITLE:

Design and simulate Smart Elevator Automation and Human Detection System using PIR Sensor. The system automatically detects human presence and controls elevator operations for improved safety and automation.

TEAM MEMBERS:

HARISANKARAN S

(24MER013)

MUTHUKUMAR M

(24MER034)

MONESHWARAN C

(24MER032)

PROJECT OVERVIEW

Elevators are widely used in residential buildings, commercial buildings, hospitals, industries, and educational institutions. Automatic detection of human presence can improve elevator operation, reduce unnecessary movement, and provide better safety and energy management. The proposed system uses a **PIR sensor, microcontroller, motor/driver, floor sensors, display, and buzzer** to detect human presence and control elevator operations automatically. The PIR sensor detects a person near or inside the elevator, and the controller operates the elevator door, motor, and floor-selection system.

PROBLEM STATEMENT

- Manual elevator operation
- Elevator movement when no person is present
- Unnecessary energy consumption
- Difficulty in automatic human detection
- Lack of automatic door control
- Delayed response to passenger presence
- Need for improved elevator safety and automation

PROPOSED SOLUTION

The PIR sensor detects human presence and sends a signal to the microcontroller. When a person is detected, the controller activates the elevator system. It controls:

- Elevator activation
- Door opening and closing
- Elevator movement
- Floor detection
- Human-presence monitoring
- Buzzer/LED safety indication

When no human presence is detected, the elevator can remain in **IDLE/STANDBY** mode.

WORKING PRINCIPLE

- The **PIR sensor** detects infrared radiation from a human body.
- When a person enters the detection area, the PIR sensor produces a **HIGH signal**.
- The **microcontroller** receives the signal and identifies that a person is present.
- The elevator system becomes active and the **door opens**.
- The passenger selects the required floor.
- The controller operates the **motor through the motor driver**.
- **Floor sensors** continuously detect the elevator position.
- When the required floor is reached, the motor **stops**.

- The elevator door opens and allows the passenger to exit.
- After the passenger leaves, the system checks for human presence again.
- If no person is detected, the elevator returns to **standby mode**.

SYSTEM ARCHITECTURE

Human Presence → PIR Sensor → Microcontroller → Elevator Motor/Driver → Floor Movement → Door Contro

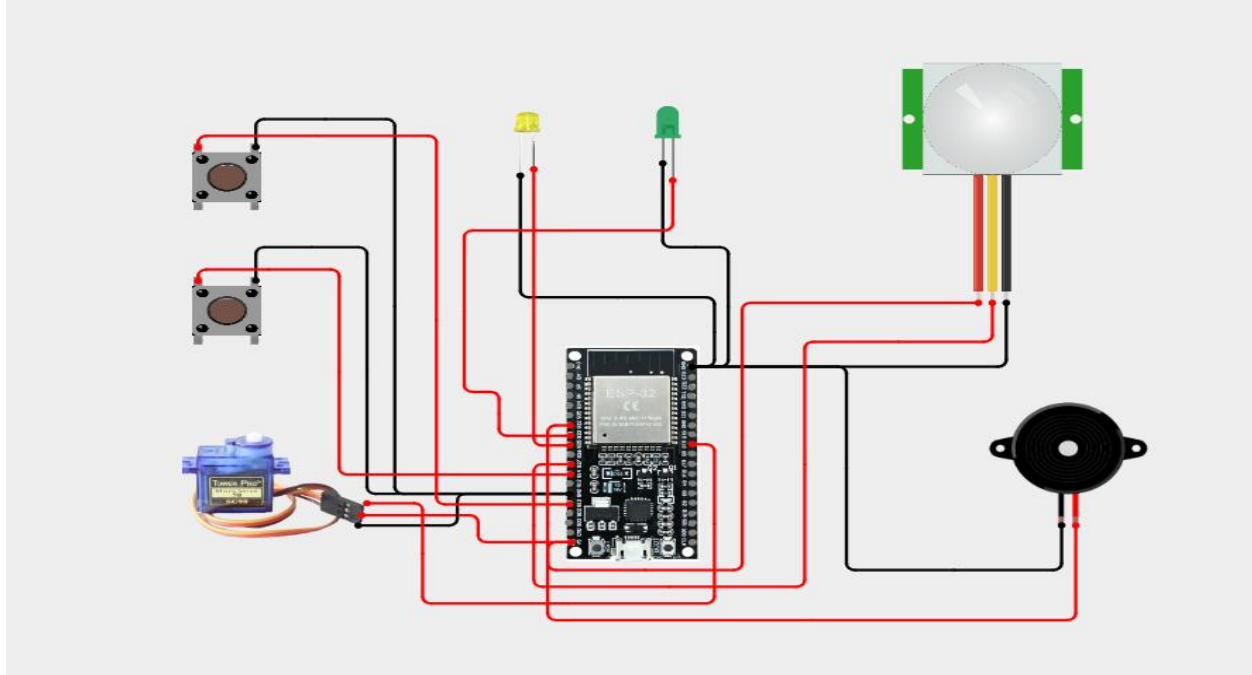
ELEVATOR OPERATING CONDITIONS

Parameter	Normal	Warning	Critical
Human Presence	Detected	—	Not detected during required operation
Elevator Position	Correct floor	Moving	Position error
Door Status	Closed/Open as required	Door delay	Door fault
Motor Status	Normal	Over-operation	Motor fault
PIR Detection	Person detected	Intermittent detection	Sensor failure

PARAMETERS MONITORED

Parameter	Sensor/Device	Purpose
Human Presence	PIR Sensor	Detect human movement/presence
Elevator Floor	IR/Limit Sensor	Detect elevator position
Motor Status	Motor Driver	Control elevator movement
Door Status	Limit Sensor	Monitor door position
Floor Selection	Push Buttons	Select destination floor
Elevator Status	Display/LED	Show system condition
Alarm Status	Buzzer	Provide warning indication

CIRCUIT DESIGN



ALGORITHM

- **START**
- Initialize the microcontroller.
- Initialize the PIR sensor.
- Initialize the floor sensors.
- Initialize the motor driver.
- Initialize the door-control system.
- Initialize the display and buzzer.
- Check for human presence using the PIR sensor.
- If no person is detected, keep the elevator in **IDLE/STANDBY** mode.
- If a person is detected, activate the elevator system.
- Open the elevator door.
- Read the selected floor.
- Close the elevator door.
- Determine the current elevator floor.
- Run the elevator motor in the required direction.
- Continuously monitor the floor sensor.
- Stop the motor when the required floor is reached.
- Open the elevator door.
- Allow the passenger to exit.
- Check for human presence again.
- Return to standby mode when no passenger is detected.
- **STOP/REPEAT.**

CODING

```
#include <ESP8266WiFi.h>
#include <Servo.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"

/* =====
WIFI
===== */

#define WIFI_SSID    "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD "qkts5737"

/* =====
ADAFRUIT IO
===== */

#define AIO_USERNAME  "Hair_13"
#define AIO_KEY       "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"
#define AIO_SERVER    "io.adafruit.com"
#define AIO_SERVERPORT 1883

/* =====
PIN CONNECTIONS
Based on your wiring diagram
===== */

// PIR Sensor
#define PIR_PIN      D2

// Servo Motor
#define SERVO_PIN    D1

// Active LOW Buzzer
#define BUZZER_PIN   D5

// Push Buttons
#define UP_SWITCH    D6
#define DOWN_SWITCH  D7
```

```

// Status LEDs
#define GREEN_LED    D3
#define RED_LED      D4

/* =====
SERVO POSITIONS
===== */

#define DOOR_CLOSED  0
#define DOOR_OPEN    180

/* =====
OBJECTS
===== */

Servo elevatorServo;

WiFiClient client;

Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);

/* =====
ADAFRUIT IO FEEDS
===== */

Adafruit_MQTT_Publish humanFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/human"
  );

Adafruit_MQTT_Publish movementFeed =
  Adafruit_MQTT_Publish(

```

```

    &mqtt,
    AIO_USERNAME "/feeds/movement"
);

/* =====
SYSTEM VARIABLES
===== */

bool humanDetected = false;
bool elevatorBusy = false;

bool lastUpState = HIGH;
bool lastDownState = HIGH;

/* =====
WIFI CONNECTION
===== */

void connectWiFi()
{
    Serial.println();
    Serial.println("Connecting to WiFi...");

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    while (WiFi.status() != WL_CONNECTED)
    {
        delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("WiFi Connected!");

    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
}

/* =====
MQTT CONNECTION
===== */

```

```

===== */

void connectMQTT()
{
    if (mqtt.connected())
    {
        return;
    }

    Serial.println("Connecting to Adafruit IO...");

    int8_t ret;

    while ((ret = mqtt.connect()) != 0)
    {
        Serial.println(mqtt.connectErrorString(ret));

        mqtt.disconnect();

        delay(5000);

        Serial.println("Retrying Adafruit IO...");
    }

    Serial.println("Adafruit IO Connected!");
}

/* =====
SEND HUMAN STATUS
===== */

void sendHuman(const char *status)
{
    Serial.print("HUMAN: ");
    Serial.println(status);

    if (!humanFeed.publish(status))
    {
        Serial.println("Failed to publish human status!");
    }
}

```



```

}

/* =====
SEND MOVEMENT STATUS
===== */

void sendMovement(const char *status)
{
    Serial.print("MOVEMENT: ");
    Serial.println(status);

    if (!movementFeed.publish(status))
    {
        Serial.println("Failed to publish movement status!");
    }
}

/* =====
BUZZER OFF
Active LOW
LOW = ON
HIGH = OFF
===== */

void buzzerOff()
{
    digitalWrite(BUZZER_PIN, HIGH);
}

/* =====
SHORT BEEP
===== */

void shortBeep()
{
    digitalWrite(BUZZER_PIN, LOW);

    delay(200);

    digitalWrite(BUZZER_PIN, HIGH);
}

```

```

}

/* =====
OPEN DOOR
===== */

void openDoor()
{
    Serial.println("DOOR OPEN");

    sendMovement("DOOR OPEN");

    // Open servo
    elevatorServo.write(DOOR_OPEN);

    // LED status
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);

    Serial.println("Door open - warning buzzer");

    unsigned long startTime = millis();

    while (millis() - startTime < 5000)
    {
        // Buzzer ON
        digitalWrite(BUZZER_PIN, LOW);
        delay(200);

        // Buzzer OFF
        digitalWrite(BUZZER_PIN, HIGH);
        delay(800);

        // Keep MQTT alive
        if (mqtt.connected())
        {
            mqtt.processPackets(10);
        }
    }
}

```

```

// Ensure buzzer is OFF
digitalWrite(BUZZER_PIN, HIGH);

Serial.println("Door warning buzzer OFF");
}

/* =====
CLOSE DOOR
===== */

void closeDoor()
{
    Serial.println("DOOR CLOSE");

    sendMovement("DOOR CLOSE");

    // Stop buzzer immediately
    digitalWrite(BUZZER_PIN, HIGH);

    // Close servo
    elevatorServo.write(DOOR_CLOSED);

    // LED status
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);

    delay(1000);

    // Ensure buzzer remains OFF
    digitalWrite(BUZZER_PIN, HIGH);
}

/* =====
ELEVATOR UP
===== */

void elevatorUp()
{
    elevatorBusy = true;

```

```
Serial.println();
Serial.println("=====");
Serial.println(" ELEVATOR UP");
Serial.println("=====");

// 1. Door open
openDoor();

// 2. Door close
closeDoor();

// 3. Moving up
Serial.println("MOVING UP");

sendMovement("MOVING UP");

digitalWrite(GREEN_LED, HIGH);
digitalWrite(RED_LED, LOW);

buzzerOff();

// Simulated elevator movement
delay(5000);

// 4. Stopped
Serial.println("STOPPED");

sendMovement("STOPPED");

digitalWrite(GREEN_LED, LOW);
digitalWrite(RED_LED, HIGH);

buzzerOff();

delay(1000);

// 5. Destination door open
openDoor();

// 6. Destination door close
```

```

closeDoor();

// 7. Idle
Serial.println("IDLE");

sendMovement("IDLE");

elevatorBusy = false;

Serial.println("UP operation completed.");
Serial.println("Waiting for next command...");
}

/* =====
ELEVATOR DOWN
===== */

void elevatorDown()
{
    elevatorBusy = true;

    Serial.println();
    Serial.println("=====");
    Serial.println(" ELEVATOR DOWN");
    Serial.println("=====");

    // 1. Door open
    openDoor();

    // 2. Door close
    closeDoor();

    // 3. Moving down
    Serial.println("MOVING DOWN");

    sendMovement("MOVING DOWN");

    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);

```

```

buzzerOff();

// Simulated elevator movement
delay(5000);

// 4. Reached
Serial.println("REACHED");

sendMovement("REACHED");

digitalWrite(GREEN_LED, LOW);
digitalWrite(RED_LED, HIGH);

buzzerOff();

delay(1000);

// 5. Destination door open
openDoor();

// 6. Destination door close
closeDoor();

// 7. Idle
Serial.println("IDLE");

sendMovement("IDLE");

elevatorBusy = false;

Serial.println("DOWN operation completed.");
Serial.println("Waiting for next command...");
}

/* =====
  SETUP
===== */

void setup()
{

```

```

Serial.begin(115200);

delay(1000);

Serial.println();
Serial.println("=====");
Serial.println(" SMART ELEVATOR AUTOMATION");
Serial.println("=====");

/* -----
   PIN MODES
   ----- */

pinMode(PIR_PIN, INPUT);

pinMode(BUZZER_PIN, OUTPUT);

pinMode(GREEN_LED, OUTPUT);
pinMode(RED_LED, OUTPUT);

pinMode(UP_SWITCH, INPUT_PULLUP);
pinMode(DOWN_SWITCH, INPUT_PULLUP);

/* -----
   SERVO
   ----- */

elevatorServo.attach(SERVO_PIN);

// Door initially closed
elevatorServo.write(DOOR_CLOSED);

/* -----
   INITIAL OUTPUTS
   ----- */

// Buzzer OFF
digitalWrite(BUZZER_PIN, HIGH);

// Green LED OFF

```

```

digitalWrite(GREEN_LED, LOW);

// Red LED ON
digitalWrite(RED_LED, HIGH);

/* -----
  WIFI
  ----- */

connectWiFi();

/* -----
  ADAFRUIT IO
  ----- */

connectMQTT();

/* -----
  INITIAL CLOUD STATUS
  ----- */

sendHuman("NO HUMAN");
sendMovement("IDLE");

Serial.println();
Serial.println("-----");
Serial.println("SYSTEM READY");
Serial.println("Door : CLOSED");
Serial.println("Elevator : IDLE");
Serial.println("Human : NO HUMAN");
Serial.println("Buzzer : OFF");
Serial.println("Waiting for human...");
Serial.println("-----");
}

/* =====
  MAIN LOOP
  ===== */

void loop()

```



```

{
  /* -----
    WIFI CHECK
    ----- */

  if (WiFi.status() != WL_CONNECTED)
  {
    connectWiFi();
  }

  /* -----
    MQTT CHECK
    ----- */

  if (!mqtt.connected())
  {
    connectMQTT();
  }

  mqtt.processPackets(10);
  mqtt.ping();

  /* -----
    PIR HUMAN DETECTION
    ----- */

  int pirState = digitalRead(PIR_PIN);

  /* -----
    HUMAN DETECTED
    ----- */

  if (pirState == HIGH && !humanDetected)
  {
    humanDetected = true;

    Serial.println();
    Serial.println(">>> HUMAN DETECTED <<<");

    sendHuman("HUMAN DETECTED");
  }
}

```

```

// Notification beep
shortBeep();

delay(200);
}

/* -----
   NO HUMAN
   ----- */

if (pirState == LOW &&
    humanDetected &&
    !elevatorBusy)
{
    humanDetected = false;

    Serial.println(">>> NO HUMAN <<<");

    sendHuman("NO HUMAN");
}

/* -----
   BUTTON CONTROL
   Only when human is detected
   ----- */

if (humanDetected && !elevatorBusy)
{
    int upState = digitalRead(UP_SWITCH);
    int downState = digitalRead(DOWN_SWITCH);

    /* -----
       UP BUTTON
       ----- */

    if (lastUpState == HIGH && upState == LOW)
    {
        delay(50);
    }
}

```

```

if (digitalRead(UP_SWITCH) == LOW)
{
    Serial.println();
    Serial.println(">>> UP BUTTON PRESSED <<<");

    shortBeep();

    elevatorUp();
}
}

/* -----
DOWN BUTTON
----- */

if (lastDownState == HIGH && downState == LOW)
{
    delay(50);

    if (digitalRead(DOWN_SWITCH) == LOW)
    {
        Serial.println();
        Serial.println(">>> DOWN BUTTON PRESSED <<<");

        shortBeep();

        elevatorDown();
    }
}

// Save button states
lastUpState = upState;
lastDownState = downState;
}
else
{
    // Update button states
    lastUpState = digitalRead(UP_SWITCH);
    lastDownState = digitalRead(DOWN_SWITCH);
}

```

```

/* -----
SMALL LOOP DELAY
----- */

delay(50);
}

```

SYSTEM WORKING

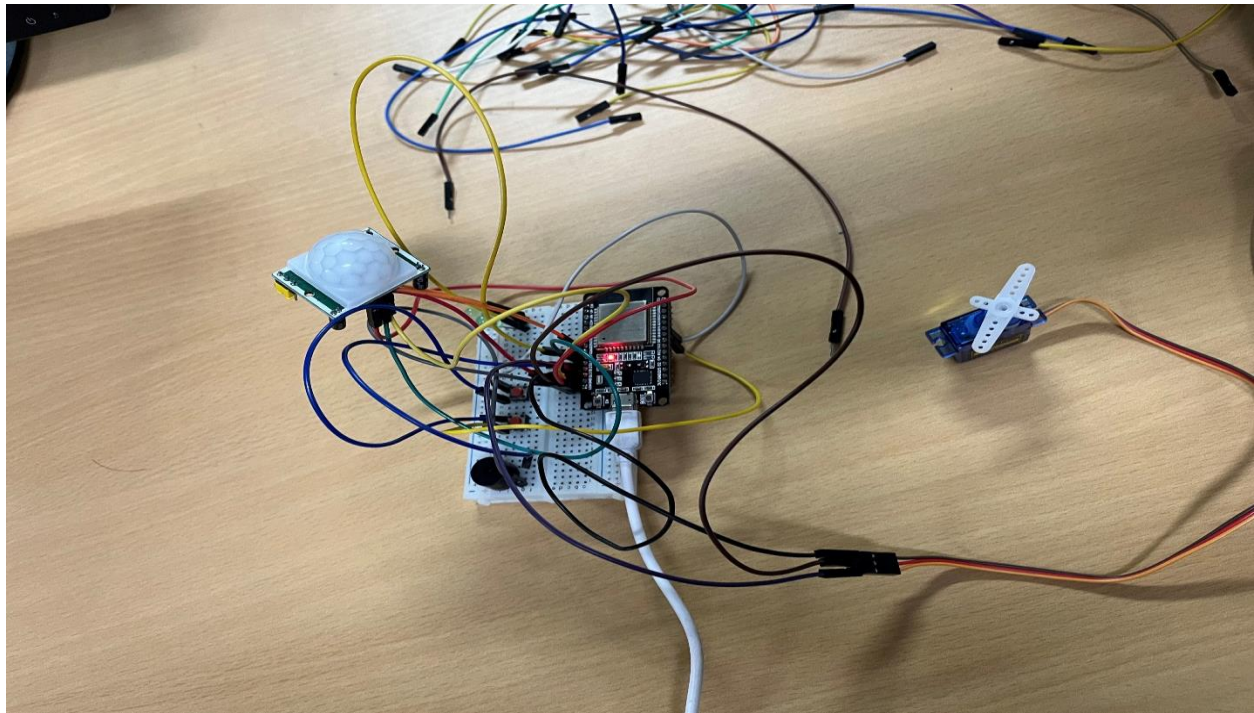
The PIR sensor continuously monitors the elevator entrance or designated detection area. When a person approaches the elevator, the PIR sensor detects human movement and sends a signal to the microcontroller. The controller activates the elevator and controls the door. After the passenger selects a destination floor, the microcontroller controls the motor through the motor driver. Floor sensors provide feedback about the elevator's position. When the elevator reaches the selected floor, the controller stops the motor and opens the door. The passenger can then leave the elevator. After the passenger leaves and no further human presence is detected, the elevator returns to its **standby condition**. This process provides automatic human detection and elevator operation.

COMPONENTS

Component	Purpose
Arduino/ESP32	Main controller
PIR Sensor	Human detection
DC Gear Motor	Elevator movement
Motor Driver	Motor control
IR/Limit Sensors	Floor detection
Push Buttons	Floor selection
Servo/DC Motor	Door operation
OLED/LCD	Elevator status
Buzzer	Safety indication
LEDs	Status indication

PROTOTYPE IMPLEMENTATION

The prototype is designed using a **microcontroller, PIR sensor, motor driver, DC motor, floor sensors, and door-control mechanism**. The PIR sensor is placed near the elevator entrance or inside the elevator cabin to detect human presence. The sensor output is connected to the microcontroller. A motor driver controls the elevator motor according to commands from the microcontroller. Floor sensors detect the position of the elevator and provide feedback to the controller. Push buttons are used to select the required floor. A display indicates the current floor and elevator status, while a buzzer provides safety or warning indications. The prototype can be simulated using **Proteus, Tinkercad, or another suitable circuit-simulation environment**, depending on the required platform



RESULTS & PERFORMANCE ANALYSIS

Condition	PIR Output	Elevator Action
No person detected	LOW	Elevator remains idle
Person detected	HIGH	Elevator becomes active
Floor selected	HIGH	Motor starts
Elevator moving	HIGH	Floor sensors monitored
Required floor reached	Detected	Motor stops
Passenger exits	Presence decreases	Door operation completed
No person present	LOW	System returns to standby

PERFORMANCE

- Automatic human presence detection
- Automatic elevator activation
- Automatic floor movement
- Automatic door control
- Real-time floor monitoring
- Reduced unnecessary elevator operation
- Improved energy efficiency
- Reduced dependence on manual operation
- Improved passenger convenience
- Safety indications through buzzer and LEDs
- Support for smart-building automation
- Low-cost prototype suitable for educational and simulation applications

